

ATIVIDADE PRÁTICA — ANÁLISE DE COMPLEXIDADE DE ALGORITMOS

Estudo de caso:

4. Parte 2 — Identificação dos algoritmos

Analise o código do projeto e identifique quais operações são realizadas.

Marque as operações encontradas:

Operação	Existe no projeto?
Inserção	☐
Remoção	☐
Busca	☐
Ordenação	☐
Alteração	☐
Percorrer toda a lista	☐
Comparação entre elementos	☐
Troca de elementos	☐
Deslocamento de elementos	☐

Depois responda:

Qual é a operação mais importante do seu projeto?

5. Parte 3 — O que significa n no seu projeto?

Esta parte é obrigatória.

Explique:

No meu projeto, o n representa...

Exemplo:

No sistema de estoque, n representa a quantidade de produtos cadastrados.

Resposta:

`\boxed{n=\text{_____}}`

6. Parte 4 — Implementando Inserção Ordenada

Agora adapte seu projeto para utilizar **Inserção Ordenada**.

Suponha que seu sistema trabalhe com códigos de produtos.

A lista deve permanecer sempre ordenada:

10

20

30

40

50

Se o usuário inserir:

35

o algoritmo deverá produzir:

10

20

30

35

40

50

Regra

O novo elemento não deve simplesmente ser colocado no final.

O algoritmo deverá:

1. localizar a posição correta;
2. deslocar os elementos necessários;
3. inserir o novo elemento.

7. Exemplo de algoritmo

Pseudocódigo:

inserirOrdenado(lista, novoElemento):

 encontrar posição correta

 enquanto elemento anterior > novoElemento:

 deslocar elemento para direita

 colocar novoElemento na posição encontrada

Por exemplo:

Lista:

10 20 40 50 60

Inserir:

30

Primeiro:

10 20 40 50 60

Deslocamos:

10 20 40 50 60

↑

Depois:

10 20 40 50 60

↑

E finalmente:

10 20 30 40 50 60

8. Parte 5 — Analisando o melhor cenário

Agora vamos analisar a eficiência.

Considere:

10 20 30 40 50 60

Queremos inserir:

70

Onde ele será colocado?

No final.

Precisamos deslocar vários elementos?

Não.

Podemos simplesmente colocar:

10 20 30 40 50 60 70

Nesse cenário, o trabalho é pequeno.

Pergunta

Por que esse é um **melhor cenário**?

Classificação aproximada

$$O(n)$$

dependendo da forma como a posição é localizada.

Se a implementação já sabe que o novo elemento será inserido no final, a movimentação propriamente dita pode ser:

$$O(1)$$

9. Parte 6 — Analisando o pior cenário

Agora considere:

10 20 30 40 50 60

Queremos inserir:

5

Onde o elemento será colocado?

No início.

Para abrir espaço, precisamos deslocar:

10

20

30

40

50

60

O resultado será:

5 10 20 30 40 50 60

Quantos elementos foram deslocados?

6

Se:

$$n = 6$$

temos aproximadamente:

n

movimentações.

Portanto:

$$O(n)$$

10. Parte 7 — Agora entra a fórmula

Suponha que a inserção necessite deslocar:

$n-1$

elementos.

Por que $n-1$?

Porque um elemento será inserido e os outros precisam abrir espaço.

Exemplo

Para:

$$n = 6$$

temos:

$$n - 1 = 5$$

possíveis deslocamentos em determinada posição.

11. Parte 8 — Melhor × pior cenário

Preencha:

Cenário	Situação	Trabalho
Melhor	inserir no final	_____
Médio	inserir no meio	_____
Pior	inserir no início	_____

Depois explique:

Por que a posição de inserção altera a eficiência do algoritmo?

12. Parte 9 — O que significa $O(n)$ no seu projeto?

Agora escreva com suas próprias palavras.

Complete:

Quando digo que a Inserção Ordenada possui comportamento $O(n)$, estou dizendo que...

Atenção

Não responda:

" $O(n)$ significa que são n operações."

A resposta esperada deve explicar **crescimento**.

Uma resposta adequada seria:

" $O(n)$ significa que, conforme a quantidade de elementos aumenta, o trabalho necessário cresce aproximadamente na mesma proporção."

13. Parte 10 — Busca Linear

Agora implemente ou identifique uma **Busca Linear**.

Considere:

10 20 30 40 50 60 70

Procure:

60

O algoritmo verifica:

10 → não

20 → não

30 → não

40 → não

50 → não

60 → encontrado

Foram necessárias:

6

verificações.

No pior caso:

$O(n)$

14. Parte 11 — Busca Binária

Como sua lista agora está ordenada, podemos utilizar **Busca Binária**.

Lista:

10 20 30 40 50 60 70 80

Procurar:

70

Começamos pelo meio:

10 20 30 40 | 50 | 60 70 80

70 é maior que 50.

Então descartamos a metade esquerda.

Restam:

60 70 80

Pegamos o meio:

60 | 70 | 80

Encontramos 70.

15. Por que $O(\log n)$?

Porque a cada etapa eliminamos aproximadamente metade dos elementos:

depois: n

depois: $\frac{n}{2}$

depois: $\frac{n}{4}$

depois: $\frac{n}{8}$

e assim por diante.

Representamos:

$$O(\log_2 n)$$

O 2 aparece porque estamos dividindo o problema em duas partes.

17. Parte 12 — Comparando as buscas

ETAPA 3 — IMPLEMENTAÇÃO DA INSERÇÃO ORDENADA

Aula 03

Agora começa efetivamente a alteração do projeto.

O aluno deverá implementar:

Inserção Ordenada

Exemplo:

10 20 30 40 50

Inserir:

25

Resultado:

10 20 25 30 40 50

Durante a aula

O professor acompanha:

1. definição da posição;
2. comparação;
3. deslocamento;
4. inserção;
5. teste.

Entrega 3

O aluno deverá apresentar:

- código;
- explicação do algoritmo;
- exemplo de execução;
- pelo menos 3 testes.

Testes obrigatórios

Teste 1 — inserir no final

10 20 30 40

↓

50

Teste 2 — inserir no meio

10 20 40 50

↓

30

Teste 3 — inserir no início

20 30 40 50

↓

10

Prazo

Final da Aula 05 – 24/09